

Figure 3: We treat attributes such as `Shape` and `Color` as neural operators. The operators map object representations into a visual-semantic space. We use similarity-based metric to classify objects.

for any of them. We first use a visual perception module to construct an object-based representation for a scene, and run a semantic parsing module to translate a question into an executable program. We then apply a quasi-symbolic program executor to infer the answer based on the scene representation. We use paired images, questions, and answers to jointly train the visual and language modules.

Shown in Figure 2, given an input image, the visual perception module detects objects in the scene and extracts a deep, latent representation for each of them. The semantic parsing module translates an input question in natural language into an executable program given a domain specific language (DSL). The generated programs have a hierarchical structure of symbolic, functional modules, each fulfilling a specific operation over the scene representation. The explicit program semantics enjoys compositionality, interpretability, and generalizability.

The program executor executes the program upon the derived scene representation and answers the question. Our program executor works in a symbolic and deterministic manner. This feature ensures a transparent execution trace of the program. Our program executor has a fully differentiable design w.r.t. the visual representations and the concept representations, which supports gradient-based optimization during training.

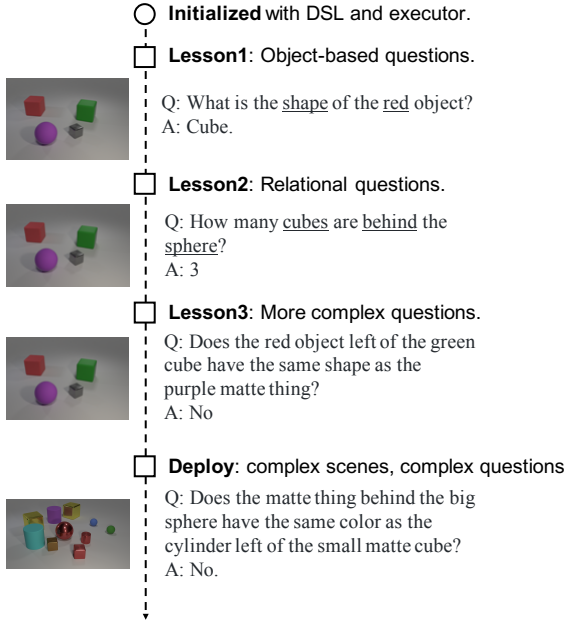
3.1 MODEL DETAILS

Visual perception. Shown in Figure 2, given the input image, we use a pretrained Mask R-CNN (He et al., 2017) to generate object proposals for all objects. The bounding box for each single object paired with the original image is then sent to a ResNet-34 (He et al., 2015) to extract the region-based (by RoI Align) and image-based features respectively. We concatenate them to represent each object. Here, the inclusion of the representation of the full scene adds the contextual information, which is essential for the inference of relative attributes such as size or spatial position.

Concept quantization. Visual reasoning requires determining an object’s attributes (e.g., its color or shape). We assume each visual attribute (e.g., shape) contains a set of visual concept (e.g., `Cube`). In NS-CL, visual attributes are implemented as neural operators, mapping the object representation into an attribute-specific embedding space. Figure 3 shows an inference an object’s shape. Visual concepts that belong to the shape attribute, including `Cube`, `Sphere` and `Cylinder`, are represented as vectors in the shape embedding space. These concept vectors are also learned along the process. We measure the cosine distances $\langle \cdot, \cdot \rangle$ between these vectors to determine the shape of the object. Specifically, we compute the probability that an object o_i is a cube by $\sigma(\langle \text{ShapeOf}(o_i), v^{\text{Cube}} \rangle - \gamma) / \tau$, where $\text{ShapeOf}(\cdot)$ denotes the neural operator, v^{Cube} the concept embedding of `Cube` and σ the Sigmoid function. γ and τ are scalar constants for scaling and shifting the values of similarities. We classify relational concepts (e.g., `Left`) between a pair of objects similarly, except that we concatenate the visual representations for both objects to form the representation of their relation.

DSL and semantic parsing. The semantic parsing module translates a natural language question into an executable program with a hierarchy of primitive operations, represented in a domain-specific language (DSL) designed for VQA. The DSL covers a set of fundamental operations for visual reasoning, such as filtering out objects with certain concepts or query the attribute of an object. The operations share the same input and output interface, and thus can be compositionally combined to form programs of any complexity. We include a complete specification of the DSL used by our framework in the Appendix A.

A. Curriculum concept learning



B. Illustrative execution of NS-CL

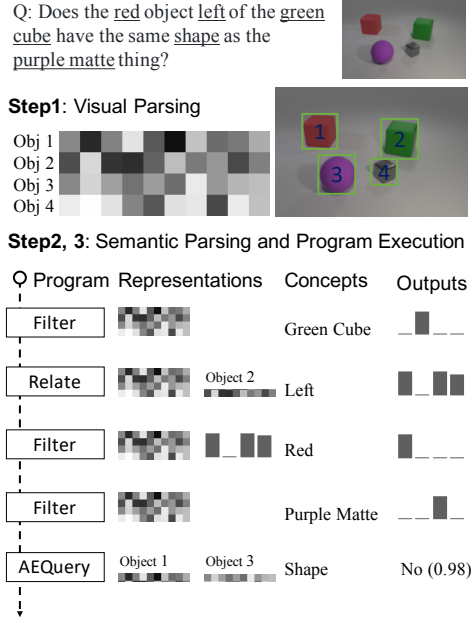


Figure 4: **A.** Demonstration of the curriculum learning of visual concepts, words, and semantic parsing of sentences by watching images and reading paired questions and answers. Scenes and questions of different complexities are illustrated to the learner in an incremental manner. **B.** Illustration of our neuro-symbolic inference model for VQA. The perception module begins with parsing visual scenes into object-based deep representations, while the semantic parser parse sentences into executable programs. A symbolic execution process bridges two modules.

Our semantic parser generates the hierarchies of latent programs in a sequence to tree manner (Dong & Lapata, 2016). We use a bidirectional GRU (Cho et al., 2014) to encode an input question, which outputs a fixed-length embedding of the question. A decoder based on GRU cells is applied to the embedding, and recovers the hierarchy of operations as the latent program. Some operations takes concepts their parameters, such as `Filter( Red )` and `Query( Shape )`. These concepts are chosen from all concepts appeared in the input question. Figure 4(B) shows an example, while more details can be found in Appendix B.

Quasi-symbolic program execution. Given the latent program recovered from the question in natural language, a symbolic program executor executes the program and derives the answer based on the object-based visual representation. Our program executor is a collection of deterministic functional modules designed to realize all logic operations specified in the DSL. Figure 4(B) shows an illustrative execution trace of a program.

To make the execution differentiable w.r.t. visual representations, we represent the intermediate results in a probabilistic manner: a set of objects is represented by a vector, as the attention mask over all objects in the scene. Each element, $Mask_i \in [0, 1]$ denotes the probability that the i -th object of the scene belongs to the set. For example, shown in Figure 4(B), the first `Filter` operation outputs a mask of length 4 (there are in total 4 objects in the scene), with each element representing the probability that the corresponding object is selected out (i.e., the probability that each object is a green cube). The output “mask” on the objects will be fed into the next module (`Relate` in this case) as input and the execution of programs continues. The last module outputs the final answer to the question. We refer interested readers to Appendix C for the implementation of all operators.

3.2 TRAINING PARADIGM

Optimization objective. The optimization objective of NS-CL is composed of two parts: concept learning and language understanding. Our goal is to find the optimal parameters Θ_v of the visual

perception module Perception (including the ResNet-34 for extracting object features, attribute operators, and concept embeddings) and Θ_s of the semantic parsing module SemanticParse, to maximize the likelihood of answering the question Q correctly:

$$\Theta_v, \Theta_s \leftarrow \arg \max_{\Theta_v, \Theta_s} \mathbb{E}_P[\Pr[A = \text{Executor}(\text{Perception}(S; \Theta_v), P)]], \quad (1)$$

where P denotes the program, A the answer, S the scene, and Executor the quasi-symbolic executor. The expectation is taken over $P \sim \text{SemanticParse}(Q; \Theta_s)$.

Recall the program executor is fully differentiable w.r.t. the visual representation. We compute the gradient w.r.t. Θ_v as $\nabla_{\Theta_v} \mathbb{E}_P[D_{\text{KL}}(\text{Executor}(\text{Perception}(S; \Theta_v), P) \| A)]$. We use REINFORCE (Williams, 1992) to optimize the semantic parser Θ_s : $\nabla_{\Theta_s} = \mathbb{E}_P[r \cdot \log \Pr[P = \text{SemanticParse}(Q; \Theta_s)]]$, where the reward $r = 1$ if the answer is correct and 0 otherwise. We also use off-policy search to reduce the variance of REINFORCE, the detail of which can be found in Appendix D.

Curriculum visual concept learning. Motivated by human concept learning as in Figure 1, we employ a curriculum learning approach to help joint optimization. We heuristically split the training samples into four stages (Figure 4(A)): first, learning object-level visual concepts; second, learning relational questions; third, learning more complex questions with perception modules fixed; fourth, joint fine-tuning of all modules. We found that this is essential to the learning of our neuro-symbolic concept learner. We include more technical details in Appendix E.

4 EXPERIMENTS

We demonstrate the following advantages of our NS-CL. First, it learns visual concepts with remarkable accuracy; second, it allows data-efficient visual reasoning on the CLEVR dataset (Johnson et al., 2017a); third, it generalizes well to new attributes, visual composition, and language domains.

We train NS-CL on 5K images (<10% of CLEVR’s 70K training images). We generate 20 questions for each image for the entire curriculum learning process. The Mask R-CNN module is pretrained on 4K generated CLEVR images with bounding box annotations, following Yi et al. (2018).

4.1 VISUAL CONCEPT LEARNING

Classification-based concept evaluation. Our model treats attributes as neural operators that map latent object representations into an attribute-specific embedding space (Figure 3). We evaluate the concept quantization of objects in the CLEVR validation split. Our model can achieve near perfect classification accuracy ($\sim 99\%$) for all object properties, suggesting it effectively learns generic concept representations. The result for spatial relations is relatively lower, because CLEVR does not have direct queries on the spatial relation between objects. Thus, spatial relation concepts can only be learned indirectly.

Count-based concept evaluation. The SOTA methods do not provide interpretable representation on individual objects (Johnson et al., 2017a; Hudson & Manning, 2018; Mascharka et al., 2018). To evaluate the visual concepts learned by such models, we generate a synthetic question set. The diagnostic question set contains simple questions as the following form: “How many *red* objects are there?”. We evaluate the performance on all concepts appeared in the CLEVR dataset.

Table 2 summarizes the results compared with strong baselines, including methods based on convolutional features (Johnson et al., 2017b) and those based on neural attentions (Mascharka et al., 2018; Hudson & Manning, 2018). Our approach outperforms IEP by a significant margin (8%) and attention-based baselines by $>2\%$, suggesting object-based visual representations and symbolic reasoning helps to interpret visual concepts.

4.2 DATA-EFFICIENT AND INTERPRETABLE VISUAL REASONING

NS-CL jointly learns visual concepts, words and semantic parsing by watching images and reading paired questions and answers. It can be directly applied to VQA.